

Đề bài chuẩn — Seminar: Apache Flink (Nhóm DP1)

Môn: CSC14118 — Big Data

0. Mục tiêu

Đạt điểm tối đa trên 5 tiêu chí rubric:

Tiêu chí	Điểm	Ghi chú
Presentation (trình bày)	20	Đúng 30 phút, rõ ràng, mọi thành viên tham gia
Report	20	10–15 trang, đúng format, không sao chép
Slide	25	Trực quan, tương phản tốt, hình chất lượng cao
Demo	25	Kịch bản thực tế, không có bước cài đặt
Class evaluation	10	Tương tác Q&A tốt, người nghe đánh giá cao

1. Nội dung bắt buộc (theo mục C — chủ đề 1 công cụ)

1.1. Tổng quan về Apache Flink

- Flink là gì: stream & batch processing engine, low-latency, high-throughput
- Mục đích thiết kế: xử lý dữ liệu thời gian thực với exactly-once semantics, event-time processing
- Mức độ phổ biến trong cộng đồng Big Data (số liệu adopters, công ty sử dụng: Alibaba, Uber, Netflix...)
- Mã nguồn mở: có (Apache License 2.0)
- Chính sách giá: miễn phí (open-source), chi phí thực tế nằm ở hạ tầng vận hành/managed service (nếu có, ví dụ Ververica Cloud, AWS Managed Flink)

1.2. Kiến trúc & chức năng

- Kiến trúc tổng quan: JobManager, TaskManager, ResourceManager, Dispatcher
- Mô hình xử lý: DataStream API (unbounded) vs DataSet/batch, Table API & SQL
- Cơ chế quan trọng: state management, checkpointing (đảm bảo fault-tolerance), watermark & event-time processing
- Tích hợp: Kafka, HDFS, JDBC connectors
- ⚠ Không đi sâu liệt kê từng hàm API — tập trung vào cách các thành phần phối hợp

1.3. So sánh với ≥ 2 công cụ tương đương

Gợi ý 2 công cụ: **Apache Spark Structured Streaming** và **Apache Kafka Streams** (hoặc Apache Storm)

So sánh trên các khía cạnh:

- Latency (true streaming vs micro-batch)
- Fault-tolerance model

- State management
- Ease of deployment / learning curve
- Use case phù hợp cho từng công cụ

1.4. Demo

- Chọn 1 kịch bản thực tế, ví dụ: real-time fraud detection từ transaction stream, hoặc real-time analytics dashboard từ Kafka topic
- Thể hiện rõ: đọc stream → xử lý (windowing/aggregation) → output kết quả
- **Không quay/trình bày** bước cài đặt Flink, tạo tài khoản, cấu hình môi trường
- Có thể live demo hoặc video quay sẵn (chất lượng cao, chữ trên console đọc rõ)

1.5. Milestone tự kiểm tra độ sâu kiến thức (chống "nông")

Dùng để nhóm tự chấm xem đã hiểu đủ sâu hay mới dừng ở mức liệt kê. Với mỗi mục, nếu chỉ trả lời được cột "Cơ bản" thì cần đọc thêm trước khi lên slide.

Tổng quan

- ☐ **Cơ bản:** Nêu được Flink là gì, dùng để làm gì
- ☐ **Đạt yêu cầu:** Giải thích được vì sao Flink ra đời để giải quyết hạn chế gì của các hệ thống trước đó (ví dụ MapReduce/Spark batch không đáp ứng tốt real-time)
- ☐ **Sâu:** Nêu được bối cảnh Flink trong hệ sinh thái (dự án Stratosphere → Apache Flink), và trade-off khi chọn Flink thay vì công cụ khác trong một tình huống cụ thể

Kiến trúc & chức năng

- ☐ **Cơ bản:** Kể tên được JobManager, TaskManager, nói chung chung "quản lý job/thực thi task"
- ☐ **Đạt yêu cầu:** Giải thích được luồng dữ liệu đi qua các thành phần thế nào (job submit → task scheduling → execution), cơ chế checkpointing hoạt động ra sao (barrier, snapshot)
- ☐ **Sâu:** Trả lời được câu hỏi "điều gì xảy ra khi 1 TaskManager crash giữa chừng" hoặc "watermark xử lý late data thế nào" — tức hiểu cơ chế bên trong, không chỉ thuộc tên gọi

So sánh công cụ

- ☐ **Cơ bản:** Liệt kê điểm khác nhau dạng bảng (tên tính năng có/không)
- ☐ **Đạt yêu cầu:** Giải thích được *tại sao* có sự khác biệt đó (ví dụ vì sao Spark Streaming có latency cao hơn — do micro-batch model)
- ☐ **Sâu:** Đưa ra được tình huống thực tế nên chọn Flink thay vì Spark/Kafka Streams, và ngược lại — thể hiện hiểu trade-off chứ không chỉ so sánh tính năng

Demo

- ☐ **Cơ bản:** Demo chạy được, ra kết quả đúng
- ☐ **Đạt yêu cầu:** Giải thích được từng bước xử lý trong lúc demo (window nào, aggregation gì, tại sao chọn cách đó)
- ☐ **Sâu:** Xử lý được câu hỏi "nếu dữ liệu đến trễ/mất kết nối thì sao", hoặc show được checkpoint/fault-tolerance thực tế đang hoạt động trong demo

Report

- ☐ **Cơ bản:** Viết lại đúng ý từ tài liệu chính thức (docs, blog)
- ☐ **Đạt yêu cầu:** Diễn giải lại bằng hiểu biết của nhóm, có ví dụ minh họa riêng
- ☐ **Sâu:** Có phần nhận định/đánh giá riêng của nhóm (ví dụ: hạn chế thực tế của Flink khi triển khai, không chỉ liệt kê ưu điểm)

Cách dùng: Trước khi chốt slide/report, mỗi phần nội dung nên tự trả lời được câu hỏi ở mức "Sâu" — nếu bí, đó là dấu hiệu đang nông và cần đọc thêm nguồn (docs chính thức, paper gốc, blog kỹ thuật của Ververica/Alibaba).

2. Cấu trúc Slide (25đ)

1. Trang bìa (tên nhóm, chủ đề, môn học)
2. Agenda
3. Tổng quan Flink (1–2 slide)
4. Kiến trúc & cơ chế hoạt động (2–3 slide, có diagram)
5. So sánh với công cụ khác (1–2 slide, dùng bảng/biểu đồ)
6. Demo (giới thiệu kịch bản trước khi chạy/chiếu video)
7. Kết luận & Q&A

Checklist hình thức:

- ☐ Tương phản nền/chữ rõ ràng
 - ☐ Không nhồi chữ, ưu tiên hình minh họa/diagram có nguồn gốc kỹ thuật (tránh clipart Canva vô nghĩa)
 - ☐ Số trang lớn, dễ nhìn
 - ☐ Hình ảnh độ phân giải cao
-

3. Cấu trúc Report (20đ)

Format: 10–15 trang, Times New Roman 12, giãn dòng 1.5, lề mặc định Word.

1. Giới thiệu & mục đích Flink
2. Kiến trúc hệ thống (JobManager/TaskManager, state, checkpointing)
3. Các chức năng chính (DataStream API, windowing, event-time)
4. So sánh với ≥ 2 công cụ tương đương (bảng so sánh)
5. Kịch bản demo & kết quả
6. Kết luận
7. Tài liệu tham khảo

⚠ Viết bằng lời văn của nhóm, không dịch/copy nguyên văn tài liệu gốc.

4. Timeline đề xuất

Mốc	Việc cần làm
Trước 16/08/2026	Hoàn thiện & share slide vào Google Drive nhóm
Ngày thuyết trình	Trình bày 30' + Q&A 15', tất cả thành viên có mặt
Trong vòng 1 tuần sau thuyết trình	Nộp đầy đủ: slide, report, video demo, tài liệu tham khảo, mã nguồn

5. Phân công gợi ý (theo 4 phần nội dung)

- Người 1: Tổng quan + So sánh công cụ
- Người 2: Kiến trúc & chức năng
- Người 3: Demo (code + kịch bản)
- Người 4: Report + tổng hợp slide

(Điều chỉnh theo số lượng thành viên thực tế của nhóm)